MOMENTUM

[MNTM]

Fitness & Wellness Tracker – Design Report

ECE 5010

Software Design Design Report

Group: 10
Abdul Muqtadir Mohammed - 202247524 - amuqtadirm
Kevin Joe Abraham - 202234134 - kjabraham
Ahmed Hassan - 202152179 - ahkmabdelrah

Due: March 05, 2026

Table of Contents

1 Project Overview

2 Requirements

2.1 Functional Requirements

2.2 Non-Functional Requirements (FURPS+)

3 High-Level Design and Architecture

3.1 Architectural Style: Layered + MVVM

3.2 Main Components

3.3 SOLID Principles in the Design

4 Class Diagrams

5 Key Use Cases and Sequence Diagrams

5.1 UC-2: Log a New Workout

5.2 UC-6: View Weekly Progress on Dashboard

6 Error Handling and Testing

7 Conclusion

1. Project Overview

Momentum (MNTM) is a single-user Windows desktop application built in C# using the .NET framework and Windows Presentation Foundation (WPF). It is designed for busy students and professionals who want a clean, local tool for logging workouts, tracking personal records, and reviewing training progress over time without the complexity of cloud-connected platforms or subscription-based mobile apps.

Momentum has drawn inspiration from Hevy, a popular workout logging app for strength training, with features such as exercise libraries, structured training, and personal record tracking. Both apps include an exercise library categorized by training type, personal record logging functionality, and a workout history available for review and editing. However, Hevy distinguishes itself by including a social feed, cloud syncing, and wearable device integration; limitations like this are part of a more focused scope for an academic project. Momentum incorporates local data storage via JSON files. Also, the application is fully offline and designed for Windows.

The two core questions the application answers for its users are:

- How much did I train this week? — workout count, total minutes, and active days.
- Am I improving? — personal record history per exercise and weekly totals on the dashboard.

The present implementation encompasses four functional views. These include: a Dashboard displaying real-time metrics; a Workouts view for session logging and editing; a Library view for exercise curation and personal record control; and a Settings view for user profile and training split configuration. The Goal and UserProfile model classes are stubs and will be completely developed in the subsequent development phase.

2. Requirements

Before any design or implementation decisions were made, the team established a clear set of requirements. This section documents those requirements: functional requirements describing what the system must do, and non-functional requirements using the FURPS+ classification from the course.

2.1 Functional Requirements

User Profile and Settings

- The user can enter and save a personal profile, including display name, age, height, weight, and preferred training split (PPL, Upper/Lower, Full Body, Bro Split).
- The saved profile loads automatically when the application starts.
- The training split selection configures how category tabs are displayed in the Library view.

Workout Logging and History

- The user can log a workout with type (Cardio, Strength, Flexibility), date, time, duration in minutes, and optional notes.
- The user can view the full workout history in a scrollable list ordered by date.
- The user can select any existing workout to edit its details.
- The user can delete any existing workout entry.
- All changes to workouts are persisted to local storage immediately.

Exercise Library and Personal Records

- The application provides an exercise library organized by category (Push, Pull, Legs, Cardio), each listing name, target muscle, and equipment.
- The user can search the library by exercise name, muscle group, or equipment.
- The user can filter exercises by category using the tab bar, which adapts to the chosen training split.
- The user can log a personal record (PR) for any exercise, including weight lifted, date, and notes.
- The user can update or delete any existing personal record.

Dashboard

- The dashboard displays all-time total workouts, total training time, workouts this week, distinct active days this week, and weekly duration.
- The dashboard updates automatically whenever the workout list changes, without requiring a manual refresh.

Goals and Weekly Summary — Planned for Final Demo

- The user can set a weekly goal as either a target number of workouts or a target number of minutes.
- The dashboard shows progress toward the active weekly goal as a percentage.
- The application generates a weekly summary including total workouts, total minutes, goal status (met/not met), and a feedback message.

2.2 Non-Functional Requirements (FURPS+)

The following non-functional requirements are organized using the FURPS+ model covered in the course.

Category	ID	Requirement
Usability	U1	All core actions (log workout, view history, dashboard, library) are reachable from the main sidebar with a single click.
	U2	Validation errors are shown inline with clear, plain-English messages (e.g., "Duration must be a positive number").
Reliability	R1	Data is saved to disk immediately after every change; normal application closure does not lose any workout entry.

Category	ID	Requirement
	R2	If a JSON data file is missing or empty on startup, the application falls back to an empty list rather than crashing.
Performance	P1	For up to several thousand workouts, list display and filtering respond within one second.
Supportability	S1	The design follows SOLID principles to avoid design smells (rigidity, fragility, and needless complexity) as outlined in the course notes.
	S2	Business logic lives in service classes, not in code-behind, so that logic can be tested independently of the UI.
Constraints (+)	C1	C# on .NET with WPF. No cloud backend or social features. All storage is local JSON.

3. High-Level Design and Architecture

3.1 Architectural Style: Layered + MVVM

Momentum employs a multi-layered architecture with a separation-of-concerns approach inspired by MVVM. Because of its clear division of UI code, business logic, and data, the Model-View-ViewModel (MVVM) pattern allows an application to be better tested, maintained, and extended. Since the team is new to WPF, the current approach uses code-behind files as lightweight controllers rather than creating full ViewModel classes with INotifyPropertyChanged bindings. The design has been constructed so that a complete MVVM refactor can be performed in the next phase without changing the Model or Service layers.

The four layers of the architecture and their responsibilities are:

- Presentation Layer: WPF UserControl views (DashboardView, WorkoutsView, LibraryView, SettingsView) and the MainWindow shell. Views display data and route user events only.
- Application Layer: Service classes (WorkoutFileService, ExerciseFileService, PRFileService, SettingsFileService) that implement use cases and mediate between views and the domain.
- Domain Layer: Plain C# model classes (Workout, Exercise, PersonalRecord, UserSettings, Goal, UserProfile) with no dependencies on WPF or file I/O.
- Data Layer: File service classes handle JSON persistence via System.Text.Json. Goal and UserProfile repositories are planned for the next phase.

3.2 Main Components

Domain Models

The domain layer is composed of six model classes. The workout serves as the primary data object, encapsulating type, date, duration, and notes for a single session. An exercise is a

defined movement that includes a name, category, target muscle, and associated equipment. Personal Record associates a weight, date, and notes with a named exercise. UserSettings holds the user's profile information and split plan. Goal and UserProfile are placeholders for the future goal-tracking functionality.

Service Layer

Each service class is associated with a single JSON file and a single model type, in compliance with the Single Responsibility Principle. WorkoutFileService manages workouts.json. ExerciseFileService manages exercises.json and seeds eleven default exercises on the first run. PRFileService manages prs.json. SettingsFileService manages settings.json, and seeds default settings on first run. Each service exposes only Load() and Save() methods to keep file I/O hidden from the view code.

Views and Application Shell

MainWindow acts as the application shell. At the start, it creates one instance of the WorkoutFileService and one instance of the shared ObservableCollection<Workout>, which loads the persisted workout list. Navigation is handled by content swapping: each sidebar button replaces the MainContent frame with the corresponding UserControl. Both DashboardView and WorkoutsView hold a reference to the same ObservableCollection, allowing the dashboard to be updated in accordance with the workout log via the C# Observer mechanism.

3.3 SOLID Principles in the Design

Single Responsibility Principle (SRP): Each class has exactly one reason to change. Model classes hold only data. Service classes handle only file I/O for one data type. View code-behind files handle only UI coordination. Changing the storage format from JSON to SQLite would require modifying only the service classes, leaving all model and view code untouched.

Open-Closed Principle (OCP): The application is open for extension without modifying existing code. New workout types can be added without touching the Workout model. New exercises can be added to the seed list without changing ExerciseFileService's Load/Save interface. The Goal and UserProfile stubs allow goal tracking to be added in a new GoalService class without restructuring existing code. This principle also motivates the Strategy pattern used for exercise filtering and goal progress, described below.

Liskov Substitution Principle (LSP): The planned IGoalStrategy interface will have two concrete implementations: WorkoutsPerWeekStrategy and MinutesPerWeekStrategy. Any caller that uses IGoalStrategy must behave correctly with either implementation, enforcing correct substitution through interface-based polymorphism.

Interface Segregation Principle (ISP): Separate service classes for workouts, exercises, personal records, and settings to avoid a single fat class that forces callers to depend on methods they do not use. Each view imports only the service it needs.

Dependency Inversion Principle (DIP): In Phase 2, ViewModels and services will depend on repository interfaces rather than concrete file-service classes. This allows test doubles to be substituted during unit testing without changing production code.

Strategy Pattern — Exercise Filtering

LibraryView.ApplyExerciseFilter() builds its filter predicate dynamically at runtime from two independent criteria: the selected category tab and the search text input. Each criterion is evaluated separately and combined using WPF's ICollectionView via Boolean AND. This is a direct application of the Strategy pattern: the filtering algorithm is encapsulated and can be extended with additional criteria (for example, filtering by equipment) without modifying the list-display logic. This also satisfies OCP — the LibraryView class is open to new filter strategies without modification.

Strategy Pattern — Goal Progress (Phase 2)

Weekly goal progress will be calculated using the IGoalStrategy interface with two concrete implementations. GoalService will invoke CalculateProgress() on the active strategy, with no knowledge of the goal type. Changing the goal type requires only swapping the strategy object, not modifying GoalService, which is a clean application of both OCP and the Strategy pattern.

```
public interface IGoalStrategy
{
    int CalculateProgress(int currentValue, int targetValue);
    bool IsGoalMet(int currentValue, int targetValue);
}

public class WorkoutsPerWeekStrategy : IGoalStrategy
{
    public int CalculateProgress(int current, int target)
        => (int)(100.0 * current / target);
    public bool IsGoalMet(int current, int target)
        => current >= target;
}
```

MVVM — Partially Applied

The existing architecture separates models, services, and views into distinct layers, in line with the MVVM architecture. `ObservableCollection<Workout>` provides automatic list binding between the data layer and WPF list controls, which is the core mechanism MVVM relies on. Full ViewModel classes with `INotifyPropertyChanged` and `RelayCommand` are planned for Phase 2 and can be introduced without changing any model or service class.

4. Class Diagrams

The following tables and diagrams describe the structure of each class across the three principal layers: domain model, service, and view. Class names, attributes, and methods are drawn directly from the current codebase.

4.1 Domain Model Classes

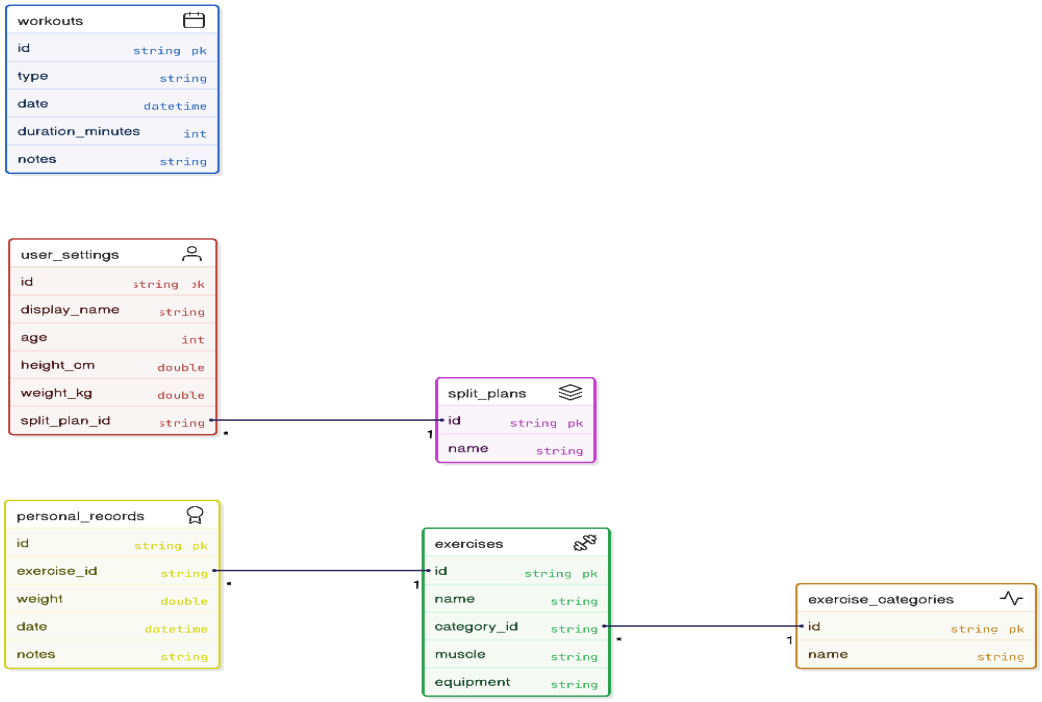


Figure 1: Domain model classes.

4.2 Service Layer Classes

Class	Responsibilities / Methods
WorkoutFileService	Manages workouts.jsonLoadWorkouts() : `List<Workout>`, `SaveWorkouts(list)`
ExerciseFileService	Manages exercises.jsonLoad() : `Load() : List<Exercise>`, `Save(list)`, `SeedExercises()` — runs on first launch
PRFileService	Manages prs.jsonLoad() : `List<PersonalRecord>`, `Save(list)`
SettingsFileService	Manages settings.jsonLoad() : `UserSettings` — seeds defaults on first run, `Save(settings)`

Table 1: Service layer classes. Each class owns exactly one JSON file and one model type (SRP).

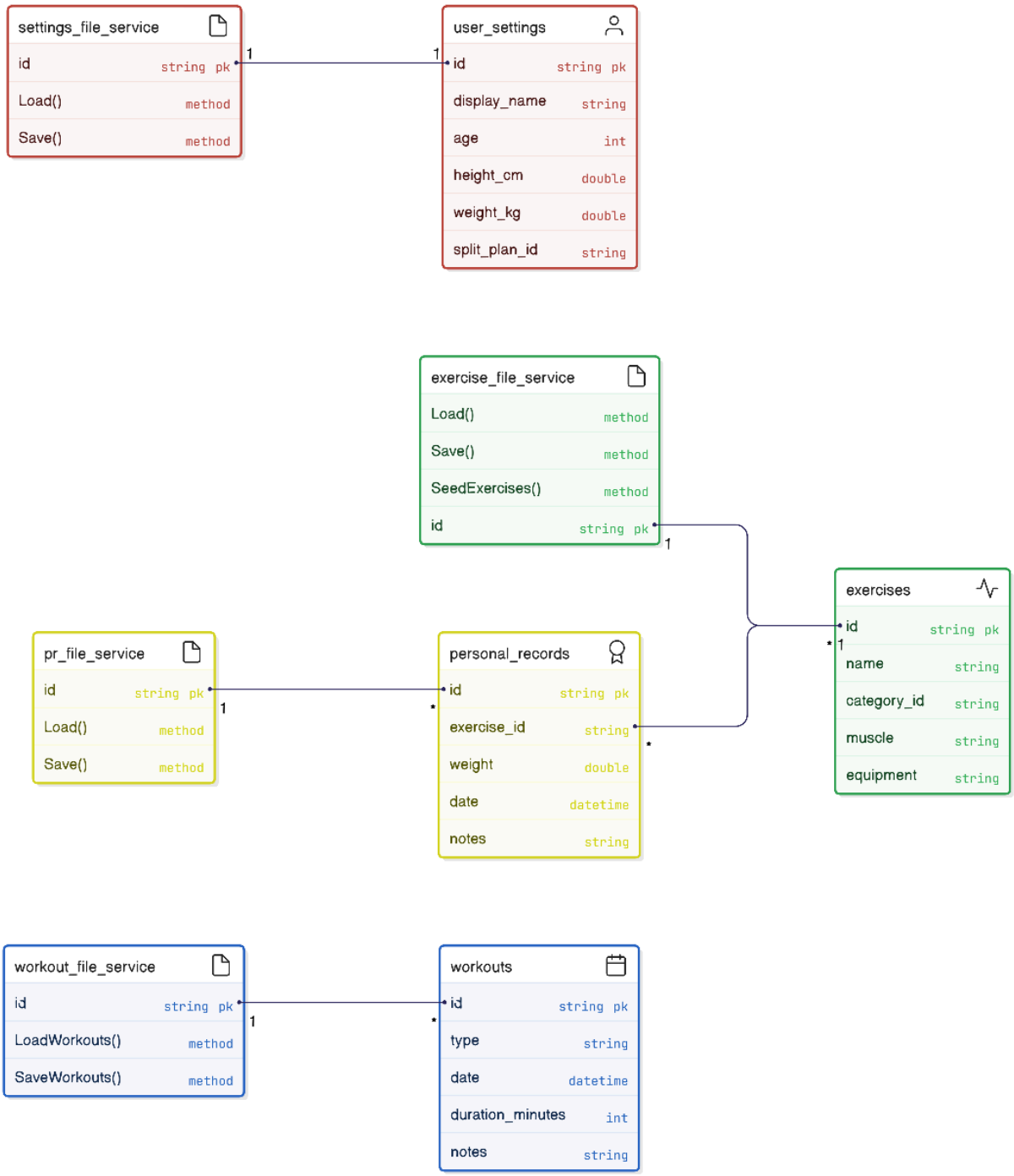


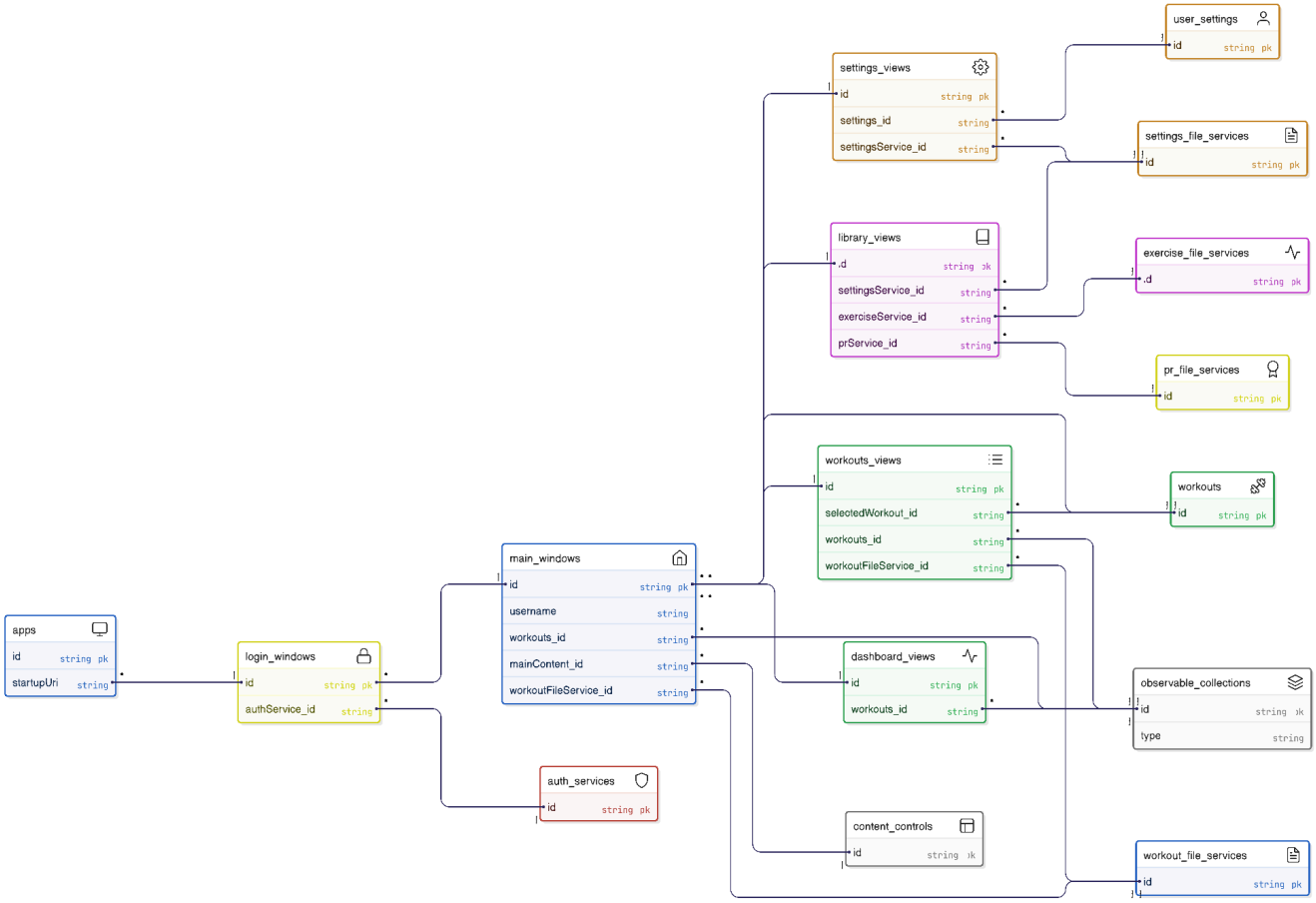
Figure 2: Service layer. Each class owns exactly one JSON file and one model type (SRP).

4.3 View Layer and Application Shell

Class	Key Responsibilities / Methods
MainWindow	Application shell — owns shared `WorkoutFileService` and `ObservableCollection<Workout>` `Dashboard_Click()`, `Workouts_Click()`, `Library_Click()`, `Settings_Click()`
DashboardView	Displays aggregate stats; auto-refreshes on any workout change `RefreshStats()`, `FormatMinutes()`, `StartOfWeek()`
WorkoutsView	Log / edit / delete workouts; writes to shared collection and calls `SaveOrUpdateButton_Click()`, `DeleteButton_Click()`, `ClearForm()`, `SaveAll()`
LibraryView	Browse exercises with search + category filter; manage personal records `ApplyExerciseFilter()`, `ConfigureTabsForSplit()`, `SavePRButton_Click()`
SettingsView	Edit and persist user profile and split `SaveButton_Click()`, `LoadintoUI()`

Table 2: View layer classes. DashboardView and WorkoutsView share the same ObservableCollection reference.

Note on the Observer relationship: DashboardView registers the handler shown below in its constructor. When WorkoutsView adds, edits, or deletes a workout, the ObservableCollection fires CollectionChanged, and DashboardView refreshes automatically without either view holding a reference to the other.



5. Key Use Cases and Sequence Diagrams

Two use cases are described in detail below. They were selected because they involve the most object interactions and best illustrate the Observer pattern and the service layer. Each diagram traces the full message chain from user input to persistence and back to the UI.

5.1 Log a New Workout

The user accesses the Workouts view and enters the workout type, date, time, duration, and optional notes before clicking Save Workout. WorkoutsView validates the data, creates a new Workout instance, adds it to the shared ObservableCollection, and invokes WorkoutFileService.SaveWorkouts() to save the current list. The DashboardView handles the ObservableCollection collection change event to call RefreshStats(), thereby automatically updating all displayed statistics.

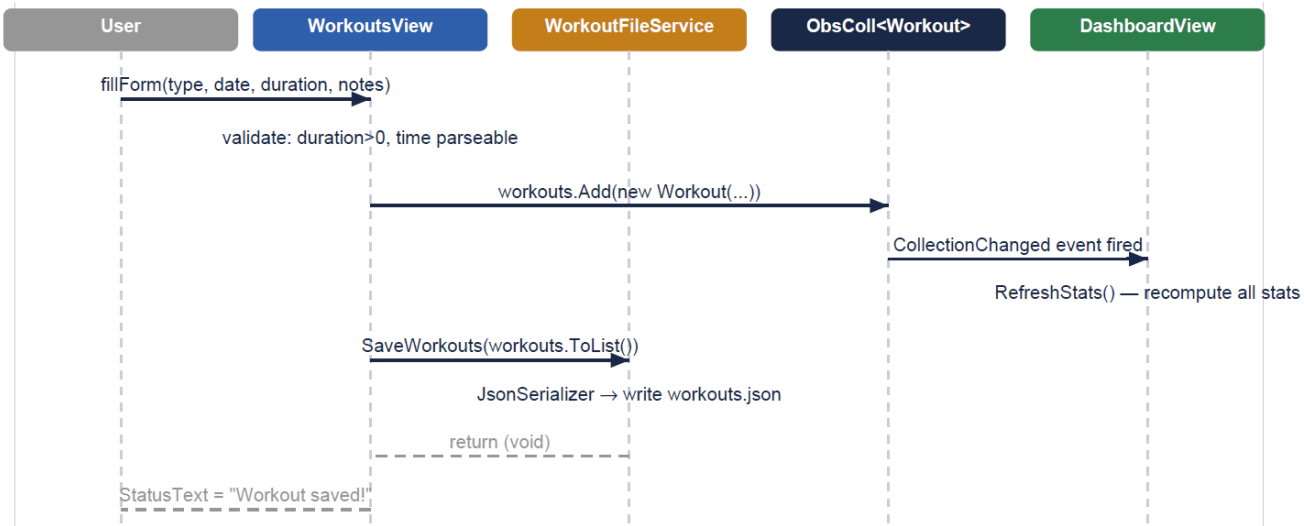
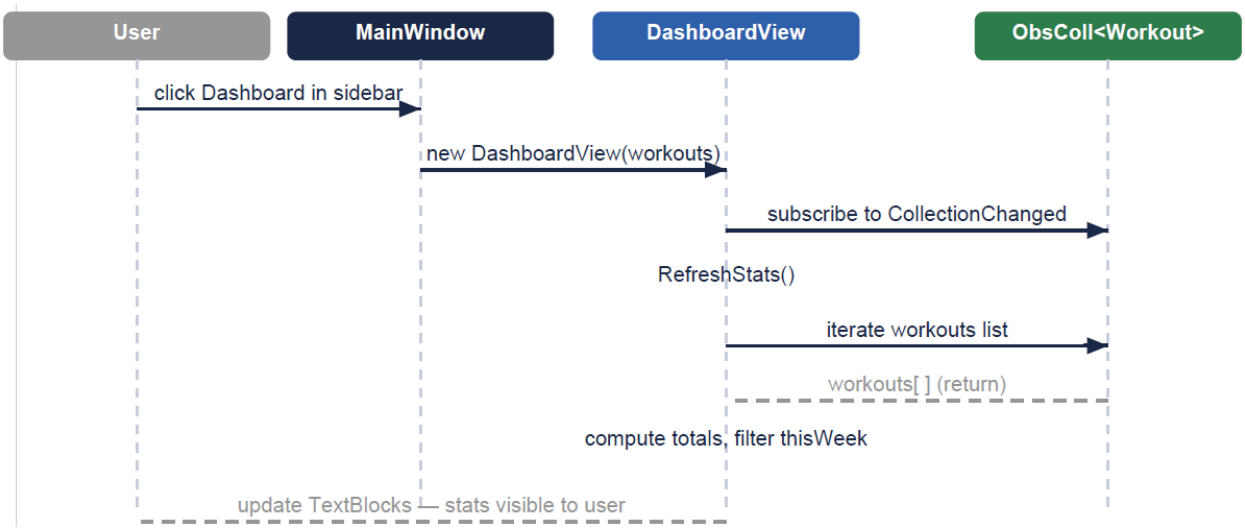


Figure 4: Sequence diagram for UC-2 — Log a New Workout.

5.2 View Weekly Progress on Dashboard

When the user clicks Dashboard in the sidebar, MainWindow creates a new DashboardView and passes the shared workout collection to it. DashboardView subscribes to the CollectionChanged event and immediately invokes RefreshStats(). This method calculates total workouts and minutes, the current week's Monday start date, workouts during the current week, the number of distinct active days, and weekly minutes. The computed values are assigned directly to WPF TextBlock controls. Any subsequent change made in WorkoutsView automatically triggers another RefreshStats() call.



Observer Pattern

This use case is a direct implementation of the Observer pattern. DashboardView is the observer; ObservableCollection<Workout> is the subject. DashboardView registers its handler in the constructor:

```
// DashboardView constructor — register as observer:
this.workouts.CollectionChanged += (_, __) => RefreshStats();
```

When WorkoutsView calls workouts.Add(), workouts[idx] = updated, or workouts.Remove() raises CollectionChanged. DashboardView is notified and refreshes automatically — without either view holding a direct reference to the other. This fully decouples the two views and is a textbook application of the Observer pattern as covered in the course.

6. Error Handling and Testing

6.1 Error Handling Strategy

Input validation occurs in the view event handlers before any model object construction or service method invocation. WorkoutsView confirms the duration is a positive integer and that the time field parses. Invalid inputs show a message in the status bar and do not continue. SettingsView applies safe defaults for any field that fails to parse. Each service class's Load() method protects against a nonexistent or empty JSON file by returning an empty collection rather than throwing an exception, ensuring the application always starts in a safe state.

Error Scenario	Response
User enters non-numeric duration	Inline status message: workout not saved
User enters an invalid time format	Inline status message: workout not saved
workouts.json is missing on startup	Empty workout list returned; no crash
exercises.json is missing on startup	Default seed library created and saved
settings.json is missing on startup	Default UserSettings created and saved
Numeric profile field fails to parse	Safe default value applied silently

Table 3: Error scenarios and application responses.

6.2 Testing Approach

The course notes describe a test-driven approach in which unit tests encode desired behaviour before implementation begins. The testing plan below follows this structure, prioritizing the service and logic layers where correctness matters most.

- Unit tests (service layer): Test WorkoutFileService, ExerciseFileService, PRFileService, and SettingsFileService in isolation with known JSON inputs. Verify Load() deserializes correctly and Save() produces valid JSON.
- Unit tests (goal logic — Phase 2): For each IGoalStrategy implementation, test CalculateProgress() and IsGoalMet() with boundary values: zero workouts, exactly at target, and above target.
- Integration tests: Verify that WorkoutsView.SaveOrUpdate correctly adds a Workout to the shared ObservableCollection, which fires the CollectionChanged event, and that DashboardView.RefreshStats() produces correct updated counts.
- Manual scenario tests: Full walkthrough of each use case on a clean install — log workout, verify dashboard updates; edit workout; change split plan; delete workout; verify JSON file contents.

7. Conclusion

This design report outlines Momentum (MNTM), a simply designed C#/WPF desktop application for tracking workouts, inspired by Hevy's design but more limited for an academic project. The design consists of a standard layered architecture with MVVM for separation of concerns, clear Single Responsibility assigned to the service layer, and the Observer pattern implemented using ObservableCollection<T> and CollectionChanged events.

The codebase at this point has working implementations of the Dashboard, logging/editing workouts, Exercise Library with personal records, and Settings views. The Goal and UserProfile classes are stubs. The biggest risks as we go forward are finishing the goal-tracking feature in the time we have left, creating a test project to confirm the correctness of the service layer, and strengthening JSON persistence to avoid edge-case issues with write interruptions.

The remaining work is to implement GoalService using the Strategy pattern, finalize the full ViewModel classes with INotifyPropertyChanged and RelayCommand, implement workout filtering, and implement the design outlined in this report for weekly summary generation. There will be no need to restructure any existing parts of the model or service layers.

For the final demonstration of the application, we will create and run multiple realistic use cases to test it to the fullest. These scenarios will include logging a new workout, editing or deleting an entry, viewing progress on the dashboard, and interactions with the exercise library and personal records. The aim is to ascertain the consistency of the application and that all its systems are cooperating. All in all, the current design allows significant work on the remaining features and provides an understandable, maintainable organizational structure.

References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- Hevy. (2019). *How we built Hevy*. Retrieved from https://www.hevyapp.com/how-we-built-hevy/ https://www.hevyapp.com/features/
- Martin, R. C. (2000). *Design Principles and Design Patterns*. Object Mentor.
- Microsoft. (n.d.-a). *Data binding and MVVM - .NET MAUI*. Microsoft Learn. Retrieved from https://learn.microsoft.com/en-us/dotnet/maui/xaml/fundamentals/mvvm
- Microsoft. (n.d.-b). *System.Text.Json Namespace*. Microsoft Learn. Retrieved from https://learn.microsoft.com/en-us/dotnet/api/system.text.json
- https://learn.microsoft.com/en-us/archive/msdn-magazine/2009/february/patterns-wpf-apps-with-the-model-view-viewmodel-design-pattern
- Smith, J. (2009). Patterns — WPF Apps With the Model-View-ViewModel Design Pattern. MSDN Magazine, Microsoft. Retrieved from https://learn.microsoft.com/en-us/archive/msdn-magazine/2009/february/patterns-wpf-apps-with-the-model-view-viewmodel-design-pattern